Codebase Telephony Integration & Costing Implementation Guide

Connecting Exotel, Airtel SIP, and Twilio directly into salesforce-ai-calling-agent/backend | 100 Calls/Day (12,000 Mins/Month)

1. Overview & Codebase Architecture Map

This guide outlines how to integrate three PSTN cloud telephony providers into our active FastAPI backend (c:\Internship\Backup\salesforce-ai-calling-agent\backend). Each approach allows customer calls to be answered directly on their regular phone receiver with **zero application installed on the customer's device**.

Target Backend File	Function & Integration Role
app/api/call_dispatch.py	Triggers outbound PSTN call dispatch via Telephony Provider API
app/services/call_dispatcher.py	Orchestrates call dispatch queuing and database status update
app/services/call_orchestrator.py	Manages Deepgram STT, Gemini LLM, and ElevenLabs/Aura TTS audio streams
app/main.py	Registers new Telephony API routers (Exotel / Airtel / Twilio)

2. Itemized Monthly Costing Matrix (100 Calls/Day = 12,000 Mins/Month)

Cost Component	Approach 1: Exotel Cloud (Rec.)	Approach 2: Airtel SIP Trunk	Approach 3: Twilio Voice API
Telephony Outbound Rate	■0.60 / min ($0.0071)	■0.30 / min ($0.0036)	■1.52 / min ($0.0180)
Monthly Trunk / License Fee	■2,000 ($23.80)	■2,500 ($30.00)	■97 ($1.15)
Total Telephony Cost	**■9,200 INR ($109.50)**	**■6,100 INR ($72.60)**	**■18,241 INR ($217.15)**
Deepgram STT (Nova-2)	■4,334 INR ($51.60)	■4,334 INR ($51.60)	■4,334 INR ($51.60)
Google Gemini 2.5 Flash LLM	■300 INR ($3.57)	■300 INR ($3.57)	■300 INR ($3.57)
TTS (Deepgram Aura / 11Labs)	■6,000 INR ($71.42)	■6,000 INR ($71.42)	■6,000 INR ($71.42)
AWS S3 Storage (3k Wavs)	■25 INR ($0.30)	■25 INR ($0.30)	■25 INR ($0.30)
TOTAL MONTHLY COST	**■19,859 INR ($236.40)**	**■16,759 INR ($199.50)**	**■28,900 INR ($344.05)**
Cost Per 4-Min Call	**■6.61 INR ($0.078)**	**■5.58 INR ($0.066)**	**■9.63 INR ($0.114)**
Effective Per Minute Cost	**~■1.65 INR / min**	**~■1.39 INR / min**	**~■2.40 INR / min**

3. Step-by-Step Codebase Integration for Each Approach

Approach 1: Exotel Voice Stream API Integration (Recommended)

New Backend File: app/api/exotel.py

1. Modify app/services/call_dispatcher.py to trigger Exotel outbound call API:
2. Exotel dials customer phone number over PSTN.
3. When customer answers, Exotel connects to WebSocket endpoint /exotel/stream/{call_id}.
4. exotel.py receives 8kHz mulaw audio chunks, converts to PCM, passes to CallOrchestrator, and streams AI speech back.

```
# app/api/exotel.py (New File) from fastapi import APIRouter, WebSocket, WebSocketDisconnect from app.services.call_orchestrator import CallOrchestrator router = APIRouter(prefix="/exotel", tags=["Exotel Telephony"]) @router.websocket("/stream/{call_id}") async def exotel_stream(websocket: WebSocket, call_id: str): await websocket.accept() orchestrator = CallOrchestrator(call_id=call_id) await orchestrator.start() try: while True: data = await websocket.receive_bytes() await orchestrator.process_audio_chunk(data) except WebSocketDisconnect: await orchestrator.close()
```

Approach 2: Airtel SIP Trunk + FreeSWITCH / LiveKit Gateway

New Backend File: app/services/freeswitch_service.py

1. Deploy FreeSWITCH Docker container listening on Airtel SIP IP.
2. call_dispatcher.py sends ESL socket command to FreeSWITCH.
3. FreeSWITCH dials customer phone number over Airtel SIP trunk.
4. FreeSWITCH pipes RTP audio stream directly into webrtc_agent.py using LiveKit SIP or ESL stream.

```
# app/services/freeswitch_service.py (New File) import httpx async def trigger_airtel_sip_call(phone_number: str, call_id: str): # Sends ESL Originate command to FreeSWITCH Docker Container url = "http://localhost:8081/freeswitch/originate" payload = {"destination": phone_number, "call_id": call_id, "gateway": "airtel_sip"} async with httpx.AsyncClient() as client: return await client.post(url, json=payload)
```

Approach 3: Twilio Voice Media Stream Integration

New Backend File: app/api/twilio.py

1. Modify call_dispatcher.py to use twilio.rest.Client.
2. Twilio fetches TwiML XML returning .
3. Twilio streams base64 8kHz mulaw audio chunks to FastAPI WebSocket.

```
# app/api/twilio.py (New File) from fastapi import APIRouter, Response, WebSocket router = APIRouter(prefix="/twilio", tags=["Twilio Telephony"]) @router.post("/voice") async def twilio_voice(): xml = '' return Response(content=xml, media_type="application/xml")
```

4. Integration Summary & Next Steps

• **Exotel** is recommended for fastest implementation into salesforce-ai-calling-agent/backend with zero hardware setup (**~■1.65/min total**).
• **Airtel SIP** is recommended for maximum scale at lowest per-minute cost (**~■1.39/min total**).
• All three approaches route audio **directly to the customer's phone ear receiver** without requiring any mobile app on their device.